

How to use this. Every answer here is short enough to say out loud in under a minute, and every number in it comes from a file in the repository rather than memory. Where a question has a nastier follow-up, that follow-up is printed underneath in the shaded line, because examiners rarely stop at the first answer. If you genuinely do not know something, say so and say what you would do to find out. That reads far better than a confident guess, and this project gives you plenty of real evidence to fall back on.

01 Object detection fundamentals

Q. Why did you choose YOLOv8 rather than Faster R-CNN or DETR?

A. Speed and deployability. Faster R-CNN is a two-stage detector: it proposes regions first, then classifies them, which is accurate but heavy. We needed inference inside a phone browser, so a single-stage detector was the only realistic option. DETR is elegant but slow to converge and needs far more data than our fourteen thousand images. YOLOv8 also exports cleanly to ONNX, which was a hard requirement.

Q. Explain one-stage versus two-stage detection.

A. A two-stage detector first generates candidate regions, then runs a classifier on each one. A one-stage detector predicts boxes and classes in a single pass over the feature map. Two-stage is usually more accurate on small objects, one-stage is faster. We traded some accuracy on small damage for the ability to run on the user's own device.

Q. Is YOLOv8 anchor-based or anchor-free?

A. Anchor-free. Older YOLO versions predicted offsets from predefined anchor boxes whose sizes you had to tune to your dataset. YOLOv8 predicts box distances directly from each grid cell, which removes that tuning step and helps when object sizes vary a lot, as damage does.

Q. What loss functions does YOLOv8 use?

A. Three parts. Binary cross-entropy for classification, CloU loss for box regression, and Distribution Focal Loss, which predicts a distribution over box-edge positions instead of a single number. DFL is why the boxes are relatively well localised even on soft edges like a dent boundary.

Q. What is IoU, and what is NMS for?

A. IoU is intersection over union: the overlap of two boxes divided by their combined area. Zero means no overlap, one means identical. Non-maximum suppression uses it to clean up duplicates. A detector fires many times on the same object, so you keep the highest-confidence box and delete anything overlapping it above a threshold. We measured this directly: our second model emitted about eleven raw boxes for a single car, and without NMS every photo would have triggered a "multiple vehicles" warning.

Q. What is mAP, and what is the difference between mAP at 0.5 and 0.5 to 0.95?

A. Average precision is the area under the precision-recall curve for one class, and mAP is that averaged over classes. At 0.5 you count a detection as correct if it overlaps the ground truth by at least half. The 0.5 to 0.95 version averages that over ten thresholds up to 0.95, so it rewards tight localisation, not just finding the object. Ours are 0.732 and 0.575, and the gap between them tells you our boxes are roughly in the right place but not surgically tight.

Q. What is transfer learning, and how did you use it?

A. You start from weights trained on a large general dataset and fine-tune on your smaller specific one, so the early layers that detect edges and textures come for free. We started from YOLOv8s pretrained on COCO and fine-tuned on the combined CarDD and VehiDE damage sets at 640 pixels, seed 42, with early stopping. With fourteen thousand images, training from scratch would have badly underperformed.

Q. What augmentation did you apply, and why does it matter here?

A. The Ultralytics stack: mosaic, HSV colour jitter, flips and scaling, and in our UAE fine-tuning run we added mixup, copy-paste, small rotations and label smoothing. Augmentation matters more than usual for us because damage is visually diverse and our dataset is small. It is also the lever in our planned retrain, where zoom-out augmentation is meant to teach the model that damage can be small in frame.

Q. How do you know the model is not overfitting?

A. We used early stopping on validation loss with a patience window, so training stops when validation stops improving. That said, I would not claim we have proven the absence of overfitting, because our headline number was read from that same validation set. That is exactly the weakness we publish on our model card.

02 Our detector, specifically

Q. Walk me through your detection results.

A. Overall mAP at 0.5 is 0.732, mAP 0.5 to 0.95 is 0.575, mean precision 0.756 and mean recall 0.691. But the average hides the story. Glass shatter scores 0.995 and tire flat 0.923, because those look dramatic and consistent. Crack is our worst at 0.406 with recall of only 0.389.

Q. Why is crack so much worse than glass shatter?

A. A shattered windscreen is a large, high-contrast, unmistakable texture. A crack is a thin, low-contrast line that can be a genuine crack, a panel gap, a shadow or a reflection. It is a harder problem, it has fewer clean training examples, and thin structures suffer most from downsampling through the network. We report the per-class number rather than hiding behind the average.

Q. You said your mAP is not trustworthy. Explain.

A. Two problems, and we publish both. First, the 0.732 was measured on the validation set the model early-stopped against, so it is not a clean held-out estimate. Second, it only covers six of our eight classes. Punctured and missing part have no metrics in our committed summary at all, which means we are pricing damage types we never properly evaluated. The honest fix is a genuine test split, and it is written into our retraining notebook as an exit gate.

If pushed: "So why ship it?" Because the product never claims a clean car. A scan with no findings reads "unconfirmed, not clean" and always advises an inspection. We designed the interface around the model's weakness rather than around its best number.

Q. Your condition score changes when the photo is cropped slightly. Why?

A. Because the detector was trained on close-up damage crops, and users upload whole-car photos. On a wide shot the damage occupies few pixels, confidence collapses, and which class wins becomes close to a coin flip. We measured a roughly forty-seven point swing on the same car from a three percent crop, and the reported damage class flipped from missing part to lamp broken, which have very different value impacts.

If pushed: "Did you try to patch it?" Yes, four times, and all four failed on measurement: multi-scale fusion made the spread worse, softening the confidence gate changed nothing, threshold tuning just moved the failure around, and scoring on coverage was itself unstable. All four are written up so nobody repeats them.

Q. So how do you actually fix it?

A. Retraining, and only retraining. The notebook is written: zoom-out augmentation that synthesises wide shots from close-ups, clean-car negatives to stop false positives, an honest test split, and stability as the primary metric rather than mAP. It is blocked on one thing we cannot generate, which is labelled photographs of whole cars in UAE conditions. The tooling to build that dataset is committed, with no photos in it.

Q. What is tiling, and why did you add it?

A. We run the detector four times per photo: once on the full frame and once each on the top half, bottom left and bottom right. Damage that is too small to detect at full-frame resolution becomes larger relative to the model's input when you crop and rescale. We then fuse overlapping boxes. It measurably recovers detections a single pass misses.

Q. You fuse boxes with Weighted Box Fusion rather than NMS. What is the difference?

A. NMS picks a winner and throws the rest away. Weighted Box Fusion averages the overlapping boxes, weighted by confidence, to produce a consensus box. With tiling you get several partial views of the same damage, and averaging them gives a better box than picking one and discarding real evidence.

Q. How do detections become a single score out of 100?

A. Each finding contributes a value impact based on its class, severity and how much of the car it covers. We combine them as a probabilistic union rather than a plain sum, so overlapping damage does not double count, and we cap the total deduction at sixty-two percent because photographs alone cannot justify writing off more than that. Across multiple photos we apply a repeat discount so the same dent shot from eight angles is not charged eight times.

If pushed: "Where did the discount value come from?" A parameter sweep against fixture cases, and we say plainly that it is a calibrated hedge rather than something derived from first principles.

03 The second model and capture coaching

Q. Why do you have two detection models?

A. They answer different questions. The damage model finds damage but has no concept of a car, because it only knows eight damage classes. To tell a user "the car is cut off" or "you are too far away" you first need to locate the car, so we added a small general object detector that knows the COCO car, bus and truck classes.

Q. Why YOLOX rather than YOLOv8n, which you already knew?

A. Licensing, deliberately. YOLOv8 is AGPL, which is already why we cannot sell this as closed-source software. Adding a second AGPL model would mean a future commercial pivot has to replace two models instead of one. YOLOX from Megvii is Apache-2.0, so the new component adds no new obligation. It is three and a half megabytes and we redistribute it unmodified with its hash and copyright recorded.

Q. What is different about YOLOX architecturally?

A. It is anchor-free with a decoupled head, meaning classification and box regression have separate branches rather than sharing one, and it uses SimOTA for label assignment, which decides dynamically which predictions should be responsible for which ground-truth object. Its raw output needs manual grid decoding, so we add the grid coordinate and multiply by the stride to recover absolute boxes.

Q. How does the coach decide a photo is bad?

A. Four signals. The car's box area as a fraction of the frame tells us too far or too close. Whether the box touches the frame edge tells us the car is cut off. Laplacian variance measures blur, because a blurred image has weak second derivatives. Mean luminance catches too dark or blown out. The thresholds were calibrated by degrading a real photo and measuring: sharp scored about 4200 on blur, heavily blurred scored 13.

Q. Why does the coach not feed into the score?

A. Because it would break two guarantees we rely on. Our score is deterministic and it matches between browser and backend on all fifty-six pinned cases. A second model touching that path could break either. It also would not be honest: coaching improves the input, it does not make the detector more accurate, and blending them would let us claim an accuracy improvement we have not earned.

04 Running deep learning in the browser

Q. What is ONNX and why did you export to it?

A. ONNX is an open format for representing a trained network as a computation graph, independent of the framework that trained it. We train in PyTorch through Ultralytics and export to ONNX so the same weights can run in a browser through ONNX Runtime Web, with no Python and no server.

Q. Why run inference on the device instead of the server?

A. Three reasons, and privacy is the one users care about: photographs never leave the phone, and we enforce that with a test that fails if a photo ever appears in an outbound payload. Second, cost, because our entire stack is free tier and we have no GPU budget. Third, it scales for free, since every user brings their own compute.

If pushed: "What is the cost of that choice?" A forty-three megabyte model download on first scan, and slower inference than a server GPU. We lazy-load it so nobody pays that cost unless they actually scan.

Q. Which execution provider, and why not WebGPU?

A. WASM, single-threaded. Multi-threaded WASM needs cross-origin isolation headers that would complicate the deployment, and WebGPU support is still uneven across the phones our users actually have. WASM is slower but it runs everywhere, which matters more for us than peak speed.

Q. The same photo used to produce different scores. What was happening?

A. This is my favourite bug in the project. We were resizing the image with the canvas API using high-quality smoothing, which is GPU-accelerated and not bit-stable between calls. Tiny differences in the resized pixels moved detections across the confidence threshold, which changed the score. We replaced it with a pure JavaScript area-average resampler and pinned it with a determinism test. The model was never the problem, and we had assumed it was.

Q. Did you quantise the model?

A. No. We considered it, and it would cut the download substantially, but we have not measured the accuracy cost, and shipping a smaller model whose accuracy we have not re-validated would contradict how we run everything else in this project. It is an honest open item rather than an oversight.

05 The pricing model

Q. Why gradient boosting rather than a neural network?

A. Tabular data with about thirteen hundred rows and seventeen mostly categorical features is exactly where gradient-boosted trees still beat deep networks. A neural network would need far more data to match it, would be harder to constrain, and would not give us clean SHAP attribution. Choosing the fashionable model here would have been a worse engineering decision.

Q. Why predict log price rather than price?

A. Car prices are right-skewed and errors are proportional rather than absolute. Being wrong by ten thousand dirhams on a four hundred thousand dirham car is very different from the same error on a thirty thousand dirham car. Training on $\log p$ makes the model optimise relative error, and we invert with $\exp m1$, which is the exact inverse.

If pushed: There is a real trap here. The corpus has its own log column computed as natural log, and pairing that with $\exp m1$ under-predicts every car by one dirham. We compute our own.

Q. What are monotone constraints and why do you need them?

A. They force the model's output to move only one way with respect to a feature. We constrain price to never increase with age or mileage. Without them the model learns quirks of the data, and a user dragging the mileage slider upward would occasionally see the price rise, which destroys trust instantly even if the average error is fine. We verify zero violations on sweeps.

Q. You use squared error, not quantile regression. Was that not a downgrade?

A. It sounds like one, and it was the opposite. We tested quantile regression, and the problem is that XGBoost's quantile objective silently ignores monotone constraints. Our shipped model was raising price with mileage on every sweep we tested while appearing correctly configured. Squared error honours the constraints exactly, and it was also more accurate on our data, so we switched and wrote it up.

Q. Explain SHAP.

A. SHAP assigns each feature a contribution to a specific prediction, derived from cooperative game theory, by asking how the prediction changes as features are added in every possible order. For trees there is an exact polynomial-time algorithm, so we are not approximating. It is what turns our output from a number into an argument: this car is worth this much, and mileage cost you this many dirhams.

Q. What is conformal prediction, and why not just use the model's confidence?

A. Conformal prediction gives you an interval with a finite-sample coverage guarantee under an exchangeability assumption, without assuming the errors are normal. We use split conformal: hold out a calibration set, measure the residual quantile, and use that as the band. We chose it because we tested the alternatives and they failed. Raw quantile regression promised eighty percent coverage and delivered 54.8. The common plus-or-minus twenty-five percent rule delivered 56.3. Conformal keeps its promise.

Q. What does "Mondrian" mean in your conformal setup?

A. It means we calibrate separately per group rather than pooling everything. We split by brand tier, luxury and mass market, because luxury cars are harder to price and a single global band was covering them only about seventy-five percent of the time while the average looked acceptable. Per-tier calibration lifts luxury coverage and widens their band honestly, which is the right answer even though it makes our luxury numbers look worse.

Q. Why report median APE instead of MAPE or RMSE?

A. Median absolute percentage error is robust to outliers, and a small corpus of scraped listings has real outliers, including mispriced and misfiled cars. We publish MAPE at 19.95 and RMSE too, so nobody can accuse us of picking only the flattering metric. The median is what we lead with because it describes the typical car.

06 Data, features and validation

Q. What did the vehicle-spec join actually do?

A. We joined each listing to its physical specification, so horsepower, torque, fuel economy, nought to a hundred, top speed and weight. Median error dropped from 15.65 percent to 13.18, a gain of 2.37 percentage points with a bootstrap confidence interval of 2.03 to 2.74. Match rate is eighty-one percent.

Q. How do you know that gain was real and not leakage?

A. A permutation control. We shuffled the spec values across nameplates, keeping the join keys and the column count identical, and re-ran everything. If the gain came from extra model capacity or leakage it would survive shuffling. It did not: the shuffled version scored minus 0.6 percentage points, meaning it actively hurt. That is what convinced us the signal was information rather than artefact.

If pushed: "Was there any price column in the spec data?" Yes, and we excluded it deliberately. The spec table has an approximate cost field, and joining a price column onto a price prediction task would be textbook target leakage. The exclusion is asserted at runtime, not just trusted.

Q. Thirteen hundred rows is very small. How do you defend that?

A. I do not defend it, I quantify it. We fitted a learning curve of the form $a \times n$ to the minus b plus c , on twenty-five seeds with a constant held-out test set, for both the old and the new feature sets on identical splits. The asymptote moved from 10.36 percent to 6.87 once specs were added. Reaching ten percent needs roughly 5,900 listings and nine percent needs about 14,000. So we can tell you exactly what more data buys, and that tuning is not the lever.

Q. Your prices are asking prices, not sale prices. Is that not a fundamental flaw?

A. It is a real limitation and we label it wherever it appears, including on the depreciation chart. Sellers typically settle below ask, so our estimates likely sit slightly high in absolute terms. Two mitigations: the model learns the relationship between features and price, and that structure is largely preserved even if the level is shifted. And the deal score compares an asking price to other asking prices, which is a like-for-like comparison. Sale-price data in this market is not publicly available to us.

Q. How is your cross-validation set up?

A. Five seeds times five-fold shuffled cross-validation, and every metric we publish is the mean over held-out folds only. We use multiple seeds because at this corpus size a single split is noise: coverage alone carries roughly a five percentage point standard deviation per split, so a single-split coverage claim would be meaningless.

07 Agents, retrieval and grounding

Q. Why an agent pipeline rather than one function?

A. Seven small steps with explicit state are far easier to test, stream and reason about than one large function. Each node does one job: intake, condition aggregation, valuation, retrieval, report, verification, confidence disclosure. It also gives the user the live trace, which is a trust feature rather than decoration.

Q. Describe your retrieval approach.

A. Hybrid. Dense sentence embeddings capture semantic similarity, BM25 captures exact term overlap like a model name, and structured similarity handles numeric closeness on year and mileage. Dense retrieval alone tends to return cars that read similarly but differ on the numbers that matter. Same-make precision at five is 1.0 on our benchmark.

Q. How do you stop the language model hallucinating a number?

A. We do not trust it. The model may only use figures from a computed evidence pack, each tagged with a citation id, and after it writes we run a deterministic verifier that re-checks every number and citation against that evidence. If anything fails, the report is rejected and a template writer produces a citation-correct report instead. Faithfulness scores 1.0, and we validate the metric itself by feeding it a deliberately corrupted report, which scores 0.00.

If pushed: "So the AI cannot hallucinate?" It absolutely can. What we guarantee is that a hallucinated number does not reach the user, because the gate is deterministic and sits after generation. That is a different and more honest claim.

Q. Why does the corrupted control matter?

A. Because a metric that always returns 1.0 is useless. If a scorer cannot fail, it is not measuring anything. Feeding it a report we broke on purpose, and seeing it score zero, is what shows the metric discriminates.

08 Engineering and evaluation method

Q. What is browser-backend parity and why does it matter?

A. The score the user sees is computed in their browser, but the backend also computes one. If they ever disagreed, a user could see one number and our server record another. We pin fifty-six fixture cases that both must score identically, and they do.

Q. How do you stop someone forging a clean scan?

A. Every scan carries provenance: the hash of the model that produced it, the preprocessing version and the inference config version. The backend holds an accepted set and rejects anything that does not match what we ship. Otherwise a crafted request could claim a perfect condition score with no scan behind it.

Q. How much of this is actually tested?

A. 142 committed tests, seventy-two backend and seventy frontend, run in continuous integration on every push. That includes guardrails, contract tests, scoring bands, cross-language parity, determinism, and pure-logic units for the newer features. One example I would point to: our financing test builds a genuine six percent reducing-balance loan, converts it to its equivalent flat rate, and requires the solver to recover 6.0000 percent.

Q. Give me an example of something you tried and rejected.

A. Several, and we write them down deliberately. The four failed fixes for framing instability. Quantile regression, which broke monotonicity. A naive maximum across photos for multi-photo scoring, which made eight distinct dents score the same as one dent photographed eight times. In each case the measurement, not the intuition, decided.

09 Product, licensing and ethics

Q. Can you sell this?

A. Not as closed-source software, not today. The damage weights derive from Ultralytics YOLOv8 under AGPL-3.0. AGPL permits charging money, but it does not permit withholding source from network users. A commercial closed-source route needs either an Ultralytics enterprise licence or a permissively licensed replacement detector. That is why the product is free and why every component we have added since is Apache licensed.

Q. Someone relies on your number and loses money. What is your position?

A. The product never presents itself as an appraisal. Every surface says it is an automated estimate, the report says a professional inspection is recommended when confidence is limited, and the financing card explicitly says it is not an offer and not advice. We also show the uncertainty band rather than a single number, which is the main structural defence against false precision.

Q. Is the financing calculator giving financial advice?

A. No, and we were careful about that. It computes a monthly payment from inputs the user chooses. It knows nothing about their salary, credit profile, employer or eligibility, it makes no recommendation, and it is labelled an indicative estimate rather than an offer. The genuinely useful thing it does is show that an advertised flat rate and the effective APR are different measures, which is a fact about UAE lending rather than advice.

Q. Do you have users?

A. No. The product is live and free, but we have no external users and no usage analytics, so I cannot claim traction. That is a real gap and I would rather say it than dress up a deployment as adoption.

Q. What would you do next with three more months?

A. In order: collect and label whole-car UAE photographs and run the retrain, because framing instability is our biggest technical weakness and everything else is downstream of it. Grow the corpus toward five thousand listings, which our own learning curve says is worth roughly three percentage points. Then an Arabic interface, because a UAE product without one is leaving out a large share of the market.

10 Numbers to know cold

ITEM	VALUE	WHERE IT COMES FROM
Median APE	13.18%	5 seeds × 5-fold, held-out folds only
MAPE / MAE	19.95% / AED 23,185	same run
vs make-model baseline	+52.3%	baseline MAE AED 48,652
Conformal coverage	79%	target 80%; luxury 78.2%, mass 79.3%, 20 splits
Calibration error	1.3pp	mean gap, promised vs delivered
Training rows / features	1,302 / 17	corpus now 1,307 and growing weekly
Spec join gain	+2.37pp	CI [2.03, 2.74]; permuted control −0.60pp
Spec match rate	81.0%	exact key, then nameplate-median fallback
Learning-curve floor	10.36% → 6.87%	old vs shipped feature set, paired, 25 seeds
Rows for 10% / 9%	~5,900 / ~14,000	extrapolation, indicative not promised
Detector mAP@0.5	0.732	validation subset, 6 of 8 classes, not held out

mAP@0.5:0.95	0.575	precision 0.756, recall 0.691
Best / worst class	0.995 / 0.406	glass shatter vs crack (crack recall 0.389)
Training set	14,437 + 1,184	train + val, CarDD and VehiDE, 8 classes
Framing instability	~47 points	3% crop, same car, same damage
Coach model	3.5 MB	YOLOX-Nano, Apache-2.0, advisory only
Report faithfulness	1.000	corrupted control scores 0.00
Retrieval P@5	1.000	same-make, benchmark queries
Scoring parity	56 / 56	browser equals backend
Tests	142	72 backend, 70 frontend, CI on every push

Three answers worth rehearsing until they are automatic. First, why your mAP is not a clean number, and why you publish it anyway. Second, what the permutation control proves about the spec join, because that is the strongest piece of methodology in the project. Third, what capture coaching does and does not fix. Examiners tend to probe exactly where a project is weakest, and in all three cases the honest answer is genuinely more impressive than a defensive one.